```
    register: output

  - debug:
      var: output

  - assert:
      that:
        - "web.composetest_web_1.state.running"
- "redis.composetest_redis_1.state.running"
```

The above playbook can be invoked as follows:

```
$ sudo ansible-playbook provision.yml --tags setup
```

The *docker_service* module is used to compose the services—a Web application and a Redis database server. The output of launching the containers is stored in a variable and is used to ensure that both the backend services are up and running. You can verify that the containers are running using the *docker ps* command output as shown below:

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND
CREATED             STATUS              PORTS
NAMES
03f6f6a3d48f        composetest_web     "python app.py"
18 seconds ago      Up 17 seconds       0.0.0.0:5000->5000/
tcp   composetest_web_1
fa00c70da13a        redis:alpine        "docker-
entrypoint..."   18 seconds ago      Up 17 seconds
6379/tcp                composetest_redis_1
```

Scaling

You can use the *docker_service* Ansible module to increase the number of Web services to two, as shown in the following Ansible playbook:

```
- name: Scale the web services to 2
  hosts: localhost
  connection: local
  become: true
  gather_facts: true
  tags: [scale]

  tasks:
    - docker_service:
        project_src: "/home/guest/composetest"
        scale:
          web: 2
      register: output

    - debug:
        var: output

    - name: Start container two
      docker_container:
        name: composetest_web_2
        image: composetest_web
        state: started
        ports:
          - "5001:5000"
        network_mode: bridge
        networks:
          - name: composetest_default
            ipv4_address: "172.19.0.12"
```

The above playbook can be invoked as follows:

```
$ sudo ansible-playbook provision.yml --tags scale
```

The execution of the playbook will create one more Web application server, and this will listen on Port 5001. You can verify the running containers as follows:

```
$ docker ps
CONTAINER ID   IMAGE             COMMAND      CREATED
STATUS         PORTS                  NAMES
66b59eb163c3   composetest_web "python app.py" 9 seconds ago
Up 8 seconds   0.0.0.0:5001->5000/tcp   composetest_web_2
4e8a37344598   redis:alpine  "docker-entrypoint..." 11
seconds ago    Up 10 seconds   0.0.0.0:6379->6379/tcp
composetest_redis_1
03f6f6a3d48f composetest_web   "python app.py" 55 seconds ago
Up 54 seconds  0.0.0.0:5000->5000/tcp   composetest_web_1
```

You can open another tab in the browser with the URL *http://localhost:5001* on the host system, and the text count will continue to increase if you keep refreshing the page repeatedly.

Cleaning up

You can stop and remove all the running instances. First, stop the newly created Web application, as follows:

```
$ docker stop 66b
```

You can use the following Ansible playbook to stop the containers that were started using Docker compose:

```
- name: Stop all!
  hosts: localhost
  connection: local
  become: true
  gather_facts: true
  tags: [stop]

  tasks:
    - docker_service:
        project_name: composetest
```